

Laboratorio: GitHub Copilot con Instrucciones Personalizadas en AL

Guía del Estudiante - 60 minutos

Información General

Duración: 60 minutos

Modalidad: Presencial con programación individual

Objetivo: Comparar la calidad del código AL generado por GitHub Copilot con y sin instrucciones personalizadas

Lo Que Vas a Hacer

Trabajarás con **DOS instancias de VS Code** para generar el mismo código con diferentes configuraciones:

- **Instancia A:** GitHub Copilot sin instrucciones personalizadas
- **Instancia B:** GitHub Copilot con instrucciones personalizadas para AL

Al final compararás los resultados y documentarás las diferencias.

Preparación Inicial (5 minutos)

Paso 1: Configurar Dos Proyectos

1. Crear dos carpetas:

- `AL_SinInstrucciones`
- `AL_ConInstrucciones`

2. Abrir dos instancias de VS Code:

- Una para cada carpeta
- Verificar que GitHub Copilot esté activo en ambas

3. Configurar proyectos AL básicos en ambas carpetas con estos archivos:

app.json (copiar en ambas carpetas):

json

```
{
  "id": "12345678-1234-1234-1234-123456789012",
  "name": "Laboratorio AL Copilot",
  "publisher": "Student Lab",
  "version": "1.0.0.0",
  "brief": "Laboratorio de comparación Copilot",
  "description": "Comparación de código AL con y sin instrucciones",
  "platform": "1.0.0.0",
  "application": "22.0.0.0",
  "idRanges": [
    {
      "from": 50000,
      "to": 50099
    }
  ]
}
```

Paso 2: Configurar Instrucciones (Solo en Instancia B)

En la carpeta `AL_ConInstrucciones`, crear archivo `.github/copilot-instructions.md` con este contenido:

markdown

GitHub Copilot Instructions for AL Development

Context

I'm developing extensions for Microsoft Dynamics 365 Business Central using AL. Focus on Micros

File Naming

- Use format: `ObjectName.ObjectType.al`
- Examples: `CustomerLoyalty.Table.al`, `CustomerLoyalty.PageExt.al`

AL Coding Standards

- Use PascalCase for all variables and methods
- Use 4 spaces for indentation (never tabs)
- Include DataClassification on all table fields
- Use proper field IDs starting from 50000
- Include Caption property on fields
- Use Labels for error messages instead of hardcoded text
- Add proper TableRelation for lookup fields
- Include OnValidate triggers for field validation

Code Structure

- Properties first, then fields/layout, then triggers, then methods
- Include blank line between method declarations
- Use descriptive names: `TempCustomer: Record Customer temporary;`

Error Handling

- Use `Error()` function with Label variables
- Example: `Error(EmailFormatErr)` where `EmailFormatErr: Label 'Invalid email format';`

Avoid

- Hardcoded strings in Error messages
- Missing DataClassification
- Field IDs below 50000
- Single letter variable names



Ejercicio 1: Mini-Demostración (10 minutos)

Objetivo

Ver diferencias básicas con un ejemplo simple.

Instrucciones

En **AMBAS instancias**, usar exactamente este prompt:

Crear una extensión AL para añadir campo "Manager Code" a la tabla Customer

Qué Hacer

- 1. **Escribir el prompt** en ambas instancias
- 2. **Generar código** con Copilot
- 3. **Guardar resultados** sin modificar manualmente
- 4. **Observar diferencias** iniciales

Documenta las Diferencias

Aspecto	Instancia A (Sin Instrucciones)	Instancia B (Con Instrucciones)
Nombre del archivo		
ID del campo		
Nombre del campo		
DataClassification		
Tipo de validación		
Uso de Labels		

 Ejercicio 2: Ejercicio Principal (25 minutos)

Objetivo

Crear una tabla completa y comparar resultados comprensivamente.

Especificaciones

Crear una tabla de productos con:

Campos requeridos:

- ID del producto (clave primaria, autonumeración)
- Nombre del producto (texto, obligatorio)
- Precio unitario (decimal, mayor que 0)
- Categoría (enum: Electronics, Clothing, Books, Food)
- En stock (boolean)
- Fecha de creación (automática)

Validaciones:

- Nombre no puede estar vacío

- Precio debe ser positivo
- Fecha se establece automáticamente

Prompt a Usar

En **AMBAS instancias**, usar este prompt exacto:

```
Crear tabla AL para gestionar "Customer Loyalty Cards"
```

Instrucciones de Desarrollo

1. **Tiempo:** 20 minutos para generar código en ambas instancias
 2. **Solo usar Copilot:** No hacer correcciones manuales
 3. **Si no compila:** Reformular el prompt, no editar código
 4. **Documentar iteraciones:** Cuántos intentos necesitaste
-

Ejercicio 3: Análisis Comparativo (15 minutos)

Comparación de Resultados

Usar la "**Tabla de Evaluación**" (archivo separado proporcionado por el instructor) para documentar:

- Diferencias entre ambas instancias
- Métricas de tiempo y esfuerzo
- Cálculo de ROI personal
- Reflexiones sobre aplicabilidad

Discusión Grupal

Participar en análisis conjunto de:

- Observaciones más sorprendentes
 - Patrones comunes entre estudiantes
 - Implicaciones para desarrollo profesional
-

Ejercicio 4: Exploración Creativa (5 minutos)

Objetivo

Experimentar con las capacidades avanzadas de Copilot usando las instrucciones.

Instrucciones

Usando solo la **Instancia B** (con instrucciones), experimenta con este prompt abierto:

Vamos a crear un sistema de loyalty points para Business Central.
No estoy seguro por dónde empezar... ¿tabla? ¿extensión?
¿Qué me recomiendas Copilot para un sistema de puntos de recompensa?

Qué Hacer

1. **Deja que Copilot sugiera** el approach
2. **Construye sobre sus sugerencias** con prompts de seguimiento
3. **Experimenta** con ideas que surjan
4. **Documenta** qué te sorprendió

Documenta Tus Descubrimientos

- **Sugerencia más interesante:**
- **Funcionalidad que no habías considerado:**
- **Patrón de código nuevo para ti:**

Entregables

Al finalizar, entregar:

1. **Archivos .al generados** en ambas instancias
2. **Tabla de Evaluación** completada
3. **Screenshots** de ambos códigos lado a lado

Consejos para el Éxito

Durante el Desarrollo

- **No edites código manualmente** - el punto es comparar lo que genera Copilot
- **Si algo no funciona**, reformula el prompt en lugar de corregir
- **Documenta todo** - las diferencias pequeñas pueden ser significativas
- **Mantén curiosidad** - especialmente durante la exploración final

Para el Análisis

- **Sé objetivo** en las comparaciones
 - **Focus en diferencias técnicas** medibles
 - **Considera el impacto** a largo plazo en proyectos reales
 - **Piensa como desarrollador profesional** - ¿qué código usarías en producción?
-

¡Éxito en tu laboratorio! Vas a descubrir diferencias que cambiarán cómo ves las herramientas de AI en desarrollo.